

停车场出入场系统的设计与实现指导书

一、实验目的

现代停车场普遍采用车牌自动识别技术，通过出入口的扫描设备拍摄车牌、识别信息，完成车辆的入场登记、出场计费等操作。本题要求利用网络编程技术，构建一个模拟停车场出入场系统，核心是用手机程序替代停车场的专业车牌扫描设备，通过手机拍摄/选择车牌图片，上传至服务器，服务器通过真实 OCR 技术识别车牌信息，完成完整的出入场业务逻辑，模拟真实工程场景中的网络通信与数据处理流程。

巩固网络编程核心知识点，掌握客户端-服务器架构、文件传输、HTTP/TCP 通信，结合 OCR 技术实现模拟工程场景，提升代码编写、问题调试与系统设计能力。

- 1) 熟练掌握客户端-服务器（C/S）架构的设计与实现，理解网络通信的核心原理。
- 2) 掌握 HTTP 协议的应用，能够实现客户端与服务器之间的图片文件上传、数据交互。
- 3) 了解 OCR 技术的应用，能够集成开源 OCR 工具（如 EasyOCR）实现车牌识别，提升系统的实用性；也可以使用在线的 OCR 服务（如百度 OCR）。
- 4) 掌握数据持久化存储、业务逻辑设计与异常处理方法，提升代码的健壮性与可维护性。
- 5) 培养工程实践思维，能够将网络编程、OCR 识别、移动端应用结合，解决实际场景中的简单问题。
- 6) 提升团队协作能力、文档撰写能力与问题排查能力。

三、实验内容

本实验的核心是“模拟真实停车场出入场流程”，关键要求：**手机程序替代专业扫描设备，实现车牌 OCR 识别，通过网络完成数据交互，业务逻辑完整、可正常运行**，具体要求如下：

一）功能要求（必做）

1. 手机客户端（模拟车牌扫描设备）

- 提供“拍摄车牌”功能：调用手机摄像头，拍摄清晰的车牌图片（模拟扫描动作）。
- 提供“从相册选择车牌图片”功能：支持选择本地已有的车牌图片，适配不同测试场景。
- 图片上传功能：将拍摄/选择的车牌图片，通过网络上传至服务器，触发 OCR 识别与出入场操作。
- 结果展示功能：接收服务器返回的响应数据（识别结果、入场/出场状态、停车费用等），清晰展示给用户，界面简洁、易用。

2. 服务器端（停车场管理系统）

- 图片接收功能：能够接收手机客户端上传的车牌图片文件，保存至指定目录（可定期清理）。
- OCR 车牌号识别功能：集成 EasyOCR 工具，对上传的车牌图片进行识别，提取车牌号；也可以使用在线的 OCR 服务（如百度 OCR）。
- 出入场业务逻辑：
 - 入场：识别车牌号后，记录车辆入场时间、车牌号，标记车辆状态为“在场”，返回入场成功信息至客户端。
 - 出场：识别车牌号后，校验车辆是否已入场，若已入场，计算停车时长与停车费用（费率自定，需合理），更新车辆状态为“离场”，记录出场时间，返回出场成功、停车费用等信息至客户端。
- 数据持久化功能：将车辆的出入场记录（车牌号、入场时间、出场时间、停车时长、费用、状态等）持久化存储（可选用 JSON 文件、SQLite、MySQL 等，难度自定）。
- 异常处理功能：能够处理常见异常，包括但不限于：图片上传失败、OCR 识别失败、重复入场、未入场先出场、网络中断、图片格式不支持等，返回清晰的错误提示。

3. 网络通信要求

- 通信协议：选用 HTTP 协议（推荐）或 TCP 协议，实现客户端与服务器之间的可靠通信。
- 数据格式：图片上传采用 Multipart/form-data 格式（HTTP），响应数据采用 JSON 格式，结构清晰、字段规范。
- 可访问性：服务器启动后，允许同一局域网内的手机客户端访问（支持多

设备测试)。

二) 技术要求

1. 服务器端：选用 Python 或 Java 开发（推荐 Python，上手快、易集成 OCR 工具），可使用 Flask、Django（Python）或 Spring Boot（Java）框架简化开发。
2. OCR 识别：使用 EasyOCR（开源、支持中文车牌、无需训练）或在线的 OCR 服务（如百度 OCR），实现真实的车牌识别，需对识别结果进行简单过滤（如省份简称校验、车牌号长度校验），提升识别准确率。
3. 手机客户端：可选用两种实现方式（二选一，推荐方式 1，降低开发难度）：
 - 方式 1：H5 网页版（HTML+CSS+JavaScript），可直接调用手机摄像头、相册，无需原生开发，通过浏览器访问即可使用。
 - 方式 2：Android 原生 App（Java/Kotlin 或 Python 的 Kivy 等），需实现摄像头调用、图片处理、网络请求等功能，体验更优。
4. 代码规范：代码结构清晰，命名规范（变量、函数、类名含义明确），添加必要的注释，便于阅读与调试；禁止抄袭，若发现抄袭，本次实验按 0 分处理。

三) 可选优化要求（加分项）

以下优化方向可根据自身能力选择，加分项将计入最终成绩，体现个人技术与工程思维：

1. OCR 识别优化：加入车牌区域检测（如 YOLOv5 轻量版），先定位车牌区域再进行识别；添加识别结果纠错（如区分“0”与“O”），提升识别准确率。
2. 图片预处理优化：实现图片的 OCR 预处理，减少上传的流量，加宽 OCR 速度。
3. 功能扩展：
 - 添加停车场车位管理功能，服务器记录总车位、剩余车位，入场时校验车位是否充足。
 - 开发管理员后台（网页版），支持查看所有车辆出入场记录、导出收费报表、修改停车费率等。
 - 添加车牌识别缓存功能，避免对同一图片进行重复识别，提升系统性能。
4. 性能优化：服务器端加入多线程/多进程处理，支持并发图片上传与 OCR 识别；定期清理上传的图片文件，避免磁盘占用过大；添加日志模块，记录所有

请求与异常，便于调试。

5. 客户端优化：手机客户端添加扫码动画、图片预览、识别进度提示；Android App 优化界面交互，提升用户体验。

四、实验环境与分组

分组：每 2 人一组，配合完成。

开发环境准备

1) 服务器端环境：

操作系统：Windows 10/11、macOS 或 Linux 均可。

编程语言：Python 3.8+（推荐）或 Java 8+。

依赖包（Python）：Flask（HTTP 服务器）、EasyOCR（OCR 识别）、OpenCV-Python（图片预处理）、numpy（数据处理）、pillow（图片操作）。

2) 客户端环境：

H5 版：任意手机浏览器（微信浏览器、Chrome 等）。

Android 版：Android Studio、Android 8.0+模拟器或真实手机。

3) 测试环境：

确保手机客户端与服务器在同一局域网（连接同一个 WiFi），关闭服务器防火墙，避免网络访问失败。

技术参考

- 1) Flask 框架：官方文档（<https://flask.palletsprojects.com/>），用于快速搭建 HTTP 服务器，实现图片接收与接口开发。
- 2) EasyOCR：官方文档（<https://www.jaied.ai/easyocr/>），用于实现车牌 OCR 识别，首次运行需下载中文+英文识别模型。
- 3) HTTP 文件上传：参考 Multipart/form-data 格式规范，实现图片上传功能。
- 4) H5 调用摄像头/相册：使用 HTML5 的 input 标签（accept="image/*"、capture="camera"）实现。
- 5) 数据持久化：JSON 文件（简易）、SQLite（轻量数据库）、MySQL（进阶），根据自身能力选择。

五、实验步骤

建议按以下步骤逐步开发，降低开发难度，确保功能逐步实现，避免遗漏核心需求：

步骤 1：环境搭建与基础准备

1. 搭建服务器端环境：安装 Python/Java，安装所需依赖包（如 Flask、EasyOCR），测试 EasyOCR 是否能正常运行（可先识别本地车牌图片，验证 OCR 功能）。
2. 确定客户端实现方式，搭建客户端开发环境（H5 版无需额外搭建，Android 版安装 Android Studio）。
3. 明确通信协议（HTTP/TCP），设计客户端与服务器的交互接口（如入场接口、出场接口），定义接口参数与响应格式。

步骤 2：服务器端核心功能开发

1. 实现图片接收接口：能够接收客户端上传的图片文件，保存至指定目录，处理图片格式校验、文件为空等异常。
2. 集成 EasyOCR，实现车牌识别功能：对上传的图片进行预处理（灰度化、二值化），调用 EasyOCR 识别车牌，对识别结果进行过滤（省份简称、长度校验），返回识别结果。
3. 实现数据持久化：设计数据存储格式（如 JSON 文件结构、数据库表结构），实现车辆出入场记录的保存、查询、更新功能。
4. 实现出入场业务逻辑：开发入场接口（记录入场信息）、出场接口（计算停车费用、更新离场信息），添加异常处理（重复入场、未入场先出场等）。
5. 测试服务器端接口：使用 Postman 等工具，模拟图片上传与接口调用，验证 OCR 识别、业务逻辑、异常处理是否正常。

步骤 3：手机客户端开发

1. 设计客户端界面：简洁易用，包含拍摄/选择图片按钮、入场/出场按钮、结果展示区域。
2. 实现图片获取功能：调用手机摄像头拍摄图片、从相册选择图片，预览图片。
3. 实现图片上传功能：将图片通过网络上传至服务器的对应接口，处理网络请求异常（如网络中断、请求超时）。
4. 实现结果展示功能：接收服务器返回的 JSON 响应，解析数据，以清晰的格式展示识别结果、入场/出场状态、停车费用等信息。

步骤 4：系统联调与测试

1. 局域网连接测试：确保手机客户端与服务器在同一局域网，服务器启动后，客户端能够正常访问服务器接口。
2. 功能联调：
 - 测试入场流程：手机拍摄车牌→上传→服务器识别→记录入场→客户端展示入场成功。
 - 测试出场流程：手机拍摄同一车牌→上传→服务器识别→校验入场状态→计算费用→记录出场→客户端展示出场成功与费用。
3. 异常测试：测试图片上传失败、OCR 识别失败、重复入场、未入场先出场等场景，验证异常提示是否清晰、正确。
4. 优化调整：根据测试结果，修复代码中的 bug，优化 OCR 识别准确率、网络请求速度、界面交互等。

步骤 5：文档撰写与实验提交

1. 撰写实验报告（要求见第六部分），整理开发过程、核心代码、测试结果、问题与解决方法等。
2. 整理代码文件，按规范命名，打包压缩（包含服务器端代码、客户端代码、实验报告）。
3. 按要求提交实验（提交方式、截止时间由教师另行通知）。

六、实验报告要求

实验报告是实验评分的重要组成部分，需规范、详细、清晰，体现开发思路与技术细节，报告字数不少于 3000 字，包含以下内容（可自行补充）：

- 1) 系统需求分析：
 - 功能需求：详细描述客户端、服务器端的各项功能要求。
 - 技术需求：描述开发环境、技术选型依据、核心技术难点。
- 2) 系统总体设计：
 - 系统架构图：绘制客户端-服务器架构图，说明各模块的功能与交互关系。
 - 模块划分：详细介绍客户端、服务器端的各模块功能（如图片上传模块、OCR 识别模块、业务逻辑模块等）。

- 接口设计：列出所有客户端与服务器的交互接口，说明接口地址、请求方式、参数、响应格式。
 - 数据设计：描述数据持久化的格式（如 JSON 文件结构、数据库表结构），绘制数据流程图。
- 3) 系统详细实现：
- 服务器端实现：核心代码片段（图片接收、OCR 识别、业务逻辑、异常处理），并添加详细注释说明。
 - 客户端实现：核心代码片段（图片获取、图片上传、结果展示），并添加详细注释说明。
 - OCR 识别实现：说明 EasyOCR 的集成过程、图片预处理方法、识别结果过滤逻辑。
- 4) 系统测试：
- 测试环境：说明测试所用的服务器配置、手机型号、网络环境。
 - 测试用例：设计至少 5 个测试用例（包含正常功能测试、异常测试），列出测试步骤、预期结果、实际结果。
 - 测试结果：展示系统运行的截图（客户端界面、服务器运行日志、识别结果展示），说明系统是否满足所有需求。
- 5) 问题与解决方法：开发过程中遇到的主要问题（如 OCR 识别准确率低、网络上传失败、异常处理不完善等），以及对应的解决思路、解决方法。
- 6) 系统优化与展望：说明本次实验的优化方向（已实现的加分项），以及未来可进一步完善的功能。
- 7) 总结与体会：总结本次实验的收获、遇到的困难、学到的知识与技能，以及对网络编程、OCR 技术应用的理理解。

七、注意事项

- 1) 实验需独立完成（或按小组完成，小组人数不超过 2 人，需明确分工，在报告中说明各成员职责），禁止抄袭他人代码、报告，一经发现，所有相关人员本次实验按 0 分处理，并记入课程成绩。
- 2) 严格按照实验要求实现功能，重点关注“真实 OCR 识别”，禁止通过文件名提取、手动输入等方式模拟识别，否则将扣除对应分值。
- 3) 开发过程中遇到问题，可查阅官方文档、参考教程，也可向教师、同学请教，但不得直接索要完整代码。

- 4) 按时提交实验，逾期提交将酌情扣分（逾期 1 天扣 10 分，逾期 3 天及以上按 0 分处理）。
- 5) 提交的文件需完整，包含服务器端代码、客户端代码、实验报告（Word 格式），打包压缩后命名为“学号-姓名-模拟停车场系统.rar/zip”。
- 6) 测试时确保手机与服务器在同一局域网，避免因网络问题导致系统无法正常运行；EasyOCR 模型下载较慢时，可手动下载模型文件放到指定目录。